

# Tarea 2: Números Pseudoaleatorios y Métodos de Monte Carlo

Didvin Nohel Estrada Pineda

Carné 14092

Curso Libre de Configuración 2: Análisis de Datos

Universidad del Istmo de Guatemala

Septiembre 2025

## 1. Generadores de números pseudoaleatorios

Para esta primera parte se implementaron cinco algoritmos generadores de pseudoaleatorios directamente en Python, sin apoyarse en `numpy.random` ni en el módulo `random` como motor. Cada implementación recibe su semilla como argumento explícito, y las semillas usadas en las figuras de este documento quedan documentadas en cada subsección y dentro del código fuente correspondiente en `src/prngs/`.

### 1.1. Generadores de congruencia lineal (LCG)

Un LCG produce una secuencia de enteros mediante la recurrencia

$$x_{n+1} = (ax_n + c) \text{ mód } m,$$

y normaliza cada término como  $u_n = x_n/m \in [0, 1)$ . La elección de  $a$ ,  $c$  y  $m$  determina por completo la calidad del generador: el teorema de Hull–Dobell establece que el LCG alcanza período completo  $m$  si y solo si  $\gcd(c, m) = 1$ ,  $a - 1$  es múltiplo de todos los factores primos de  $m$ , y  $a - 1$  es múltiplo de 4 cuando  $m$  es múltiplo de 4.

---

**Algorithm 1** Generador de congruencia lineal

---

**Require:** semilla  $x_0$ , multiplicador  $a$ , incremento  $c$ , módulo  $m$

```
1: for  $n = 0, 1, 2, \dots$  do  
2:    $x_{n+1} \leftarrow (a \cdot x_n + c) \text{ mód } m$   
3:    $u_{n+1} \leftarrow x_{n+1}/m$   
4: end for
```

---

Se usaron los parámetros de Numerical Recipes  $a = 1664525$ ,  $c = 1013904223$ ,  $m = 2^{32}$ , que cumplen las condiciones de Hull–Dobell y por lo tanto alcanzan período completo  $2^{32}$ . La semilla utilizada en las figuras fue  $x_0 = 20250919$ .

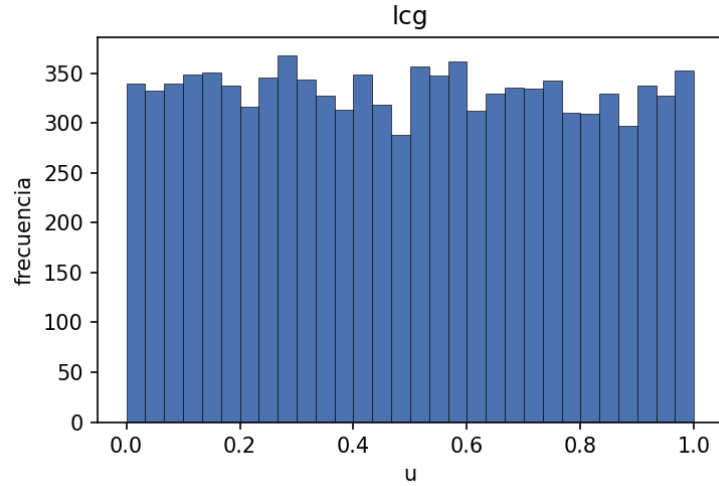


Figura 1: Histograma de 10,000 valores generados con el LCG, normalizados a  $[0, 1]$ .

Con  $N = 10,000$  valores, la prueba chi-cuadrado de bondad de ajuste (20 categorías, 19 grados de libertad) dio un estadístico  $\chi^2 = 28,39$  con  $p = 0,076$ : al ser mayor que 0,05 no se rechaza la hipótesis de uniformidad. La autocorrelación de retardo 1 fue  $r = 0,0022$  con  $p = 0,824$ , es decir, no hay evidencia de dependencia serial entre valores consecutivos.

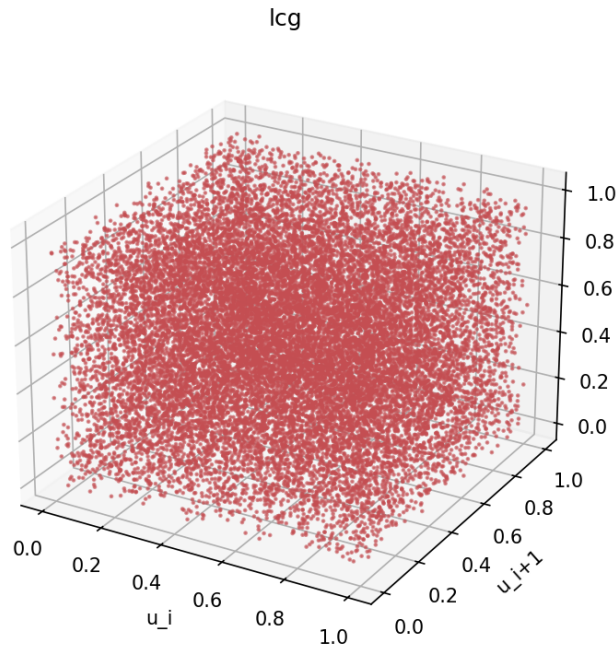


Figura 2: Ternas consecutivas  $(u_i, u_{i+1}, u_{i+2})$  del LCG en el cubo  $[0, 1]^3$ .

Las ternas del LCG llenan el cubo de manera homogénea desde cualquier ángulo de observación, sin estructura visible. Esto es consistente con haber elegido parámetros que satisfacen Hull–Dobell: un buen LCG no debería mostrar ningún patrón geométrico en la prueba espectral.

## 1.2. Método de cuadrados medios

El método de cuadrados medios, propuesto por von Neumann, eleva al cuadrado la semilla actual y extrae los dígitos centrales como el siguiente término de la secuencia. Con  $d$  dígitos por término, si  $x_n$  tiene  $d$  dígitos,  $x_n^2$  tiene hasta  $2d$  dígitos, y  $x_{n+1}$  se forma tomando los  $d$  dígitos centrales de ese cuadrado (rellenando con ceros a la izquierda si hace falta).

---

**Algorithm 2** Método de cuadrados medios

---

**Require:** semilla  $x_0$  de  $d$  dígitos

- 1: **for**  $n = 0, 1, 2, \dots$  **do**
  - 2:    $y \leftarrow x_n^2$ , escrito con  $2d$  dígitos (rellenando con ceros a la izquierda)
  - 3:    $x_{n+1} \leftarrow$  los  $d$  dígitos centrales de  $y$
  - 4:    $u_{n+1} \leftarrow x_{n+1}/10^d$
  - 5: **end for**
- 

No existe una fórmula cerrada para el período de este método: al ser un mapeo determinista sobre un conjunto finito de  $10^d$  estados, eventualmente entra en un ciclo, pero ese ciclo suele ser mucho más corto que  $10^d$ , y en muchos casos degenera rápidamente a 0 o a ciclos muy cortos. Se usó  $d = 8$  dígitos y semilla  $x_0 = 57282719$ .

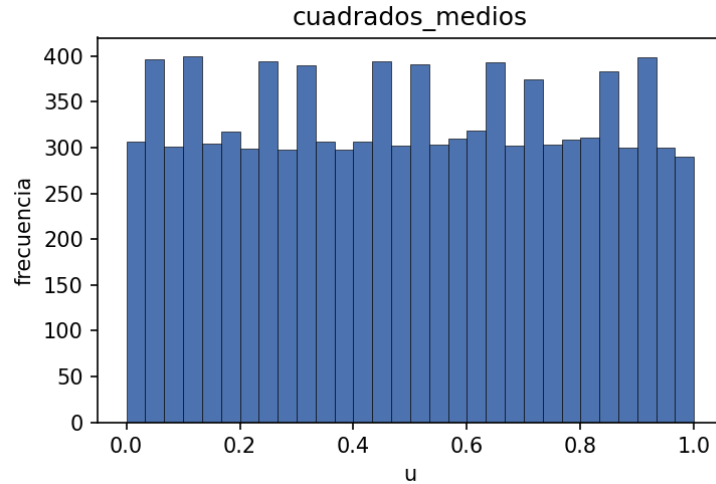


Figura 3: Histograma de 10,000 valores generados con cuadrados medios.

A simple vista el histograma parece razonablemente uniforme, pero muestra un patrón de peine (barras alternadamente más altas y más bajas). La prueba chi-cuadrado lo confirma de forma contundente:  $\chi^2 = 152,94$ ,  $p \approx 0,0000$ , rechazando uniformidad con un margen enorme. La autocorrelación de retardo 1 también resultó significativa ( $r = -0,039$ ,  $p = 0,0001$ ). Esto es exactamente la falla clásica del método: la secuencia cae en ciclos cortos que repiten un subconjunto pequeño de valores muchas veces, lo que produce un histograma visualmente aceptable pero estadísticamente muy alejado de una uniforme real.

### 1.3. Mersenne Twister

El Mersenne Twister (variante MT19937) mantiene un estado interno de  $n = 624$  palabras de 32 bits. En cada bloque de  $n$  salidas se aplica una recurrencia de "twist" sobre el estado, y cada palabra extraída pasa además por una transformación de temple (tempering) que mejora sus propiedades de equidistribución.

---

**Algorithm 3** Mersenne Twister (MT19937), esquema general

---

**Require:** semilla  $x_0$

```

1: Inicializar el arreglo de estado  $mt[0..623]$  a partir de  $x_0$  mediante un LCG auxiliar
2: for cada bloque de 624 salidas do
3:   for  $i = 0, \dots, 623$  do
4:      $y \leftarrow (mt[i] \& 0x80000000) + (mt[(i + 1) \bmod 624] \& 0x7fffffff)$ 
5:      $mt[i] \leftarrow mt[(i + 397) \bmod 624] \oplus (y \gg 1)$ 
6:     if  $y$  es impar then
7:        $mt[i] \leftarrow mt[i] \oplus 0x9908b0df$ 
8:     end if
9:   end for
10: end for
11:  $y \leftarrow mt[i]$ 
12:  $y \leftarrow y \oplus (y \gg 11)$ 
13:  $y \leftarrow y \oplus ((y \ll 7) \& 0x9d2c5680)$ 
14:  $y \leftarrow y \oplus ((y \ll 15) \& 0xefc60000)$ 
15:  $y \leftarrow y \oplus (y \gg 18)$ 
16: return  $y/2^{32}$ 

```

---

El período del Mersenne Twister es  $2^{19937} - 1$ , y su nombre viene precisamente de que ese número es un primo de Mersenne. La implementación se validó comparando la primera salida cruda con semilla 5489 contra el valor de referencia publicado, 3499211612, que coincidió exactamente.

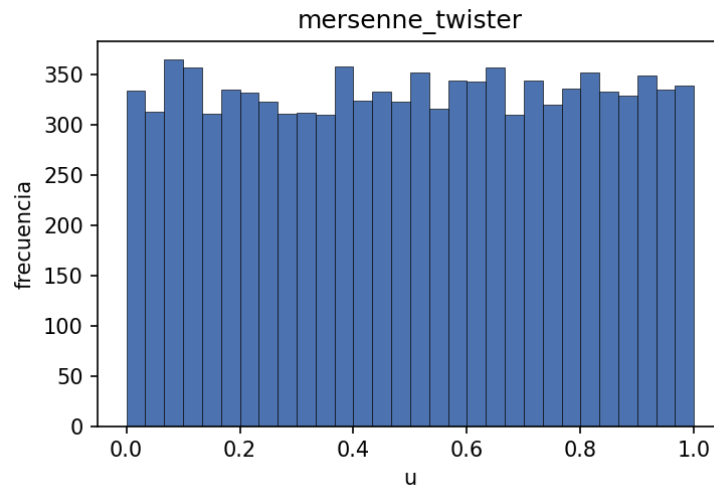


Figura 4: Histograma de 10,000 valores generados con Mersenne Twister.

Con semilla 5489 se obtuvo  $\chi^2 = 12,14$ ,  $p = 0,880$  para uniformidad, y  $r = -0,0026$ ,  $p = 0,797$

para independencia serial: ambas pruebas son plenamente consistentes con una secuencia uniforme e independiente.

## 1.4. Blum Blum Shub

Blum Blum Shub genera bits (o números) mediante  $x_{n+1} = x_n^2 \bmod M$ , donde  $M = pq$  es producto de dos primos grandes, ambos congruentes con 3 mód 4 (un entero de Blum). Su seguridad depende de la dificultad de calcular residuos cuadráticos módulo  $M$  sin conocer su factorización, un problema tan difícil como factorizar  $M$ .

---

### Algorithm 4 Blum Blum Shub

---

**Require:** semilla  $x_0$  coprime con  $M = pq$ ,  $p \equiv q \equiv 3 \pmod{4}$

```

1: for  $n = 0, 1, 2, \dots$  do
2:    $x_{n+1} \leftarrow x_n^2 \bmod M$ 
3:    $u_{n+1} \leftarrow x_{n+1}/M$ 
4: end for
```

---

Para esta tarea se usaron los primos  $p = 300007$  y  $q = 300163$  (ambos  $\equiv 3 \pmod{4}$ ), con  $M = 90,051,001,141$  y semilla  $x_0 = 987654323$ . Estos primos son mucho más pequeños que los que se usarían en una aplicación criptográfica real (donde  $p$  y  $q$  tendrían cientos de dígitos); se eligieron de este tamaño únicamente para que generar miles de valores fuera rápido en Python puro.

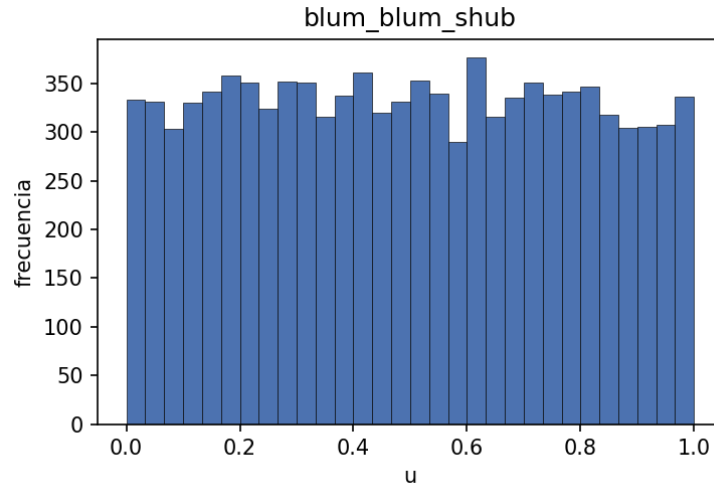


Figura 5: Histograma de 10,000 valores generados con Blum Blum Shub.

Se obtuvo  $\chi^2 = 29,66$ ,  $p = 0,056$  para uniformidad (justo por encima del umbral usual de 0.05) y  $r = 0,0174$ ,  $p = 0,083$  para independencia serial: no hay evidencia fuerte de desviación de la uniformidad ni de dependencia serial, aunque el  $p$ -valor de uniformidad queda cerca del límite.

## 1.5. RANDU

RANDU es un caso particular de LCG con  $a = 65539 = 2^{16} + 3$ ,  $c = 0$  y  $m = 2^{31}$ , usado en las computadoras IBM System/360 durante los años sesenta. Es históricamente célebre por lo mal elegido de su multiplicador.

---

**Algorithm 5** RANDU

---

**Require:** semilla impar  $x_0$

```
1: for  $n = 0, 1, 2, \dots$  do  
2:    $x_{n+1} \leftarrow (65539 \cdot x_n) \bmod 2^{31}$   
3:    $u_{n+1} \leftarrow x_{n+1}/2^{31}$   
4: end for
```

---

Debido a que  $m = 2^{31}$  y  $a - 1 = 65538$  no cumple la condición de Hull–Dobell relacionada con los factores de  $m$ , el período efectivo de RANDU con semilla impar es  $2^{29}$ , es decir, un cuarto del máximo teórico  $m$ . La semilla usada fue  $x_0 = 1$ .

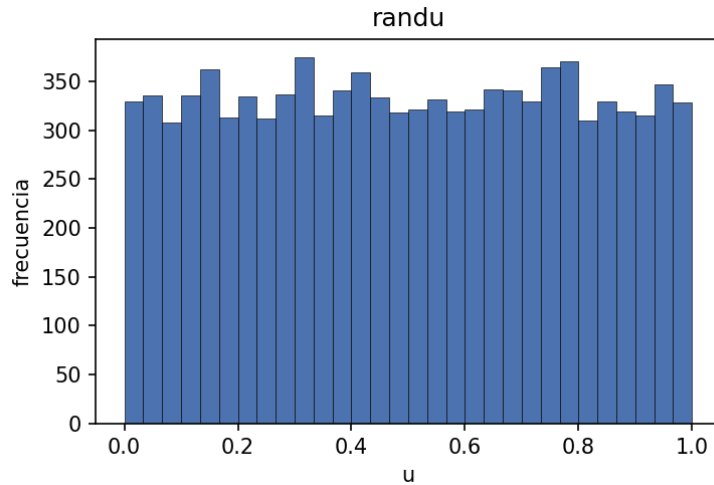


Figura 6: Histograma de 10,000 valores generados con RANDU.

En una dimensión, RANDU pasa las pruebas sin problema:  $\chi^2 = 15,16$ ,  $p = 0,712$  para uniformidad, y  $r = 0,0133$ ,  $p = 0,184$  para independencia serial. Esto ya es una lección importante: las pruebas de uniformidad e independencia serial de retardo 1 son *incapaces* de detectar el defecto real de RANDU, que solo aparece al observar ternas de valores consecutivos.

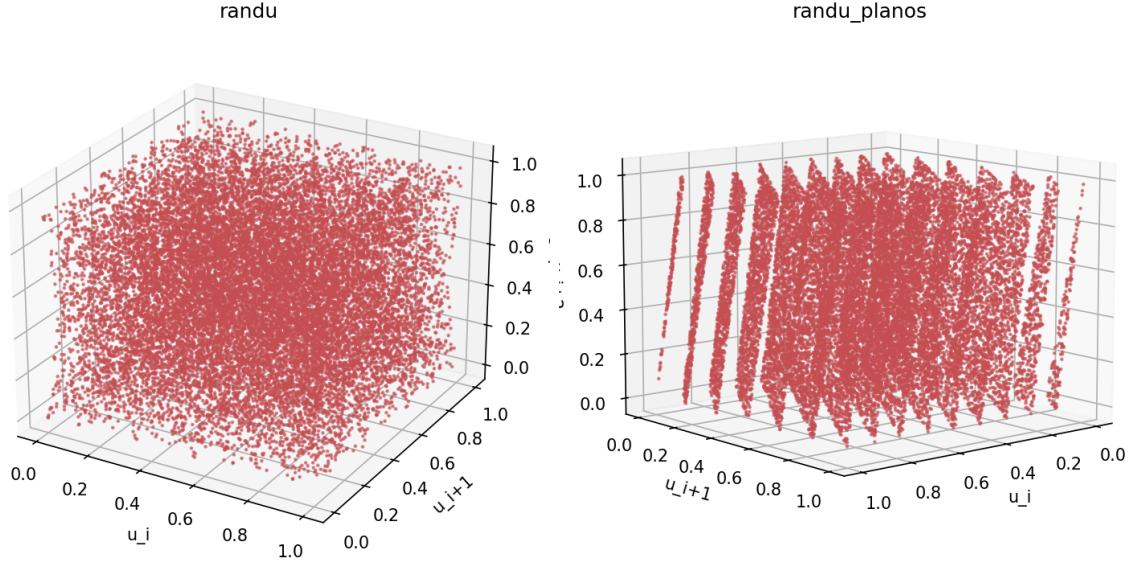


Figura 7: Ternas consecutivas de RANDU en  $[0, 1]^3$ . Izquierda: ángulo de vista genérico, donde el defecto no es evidente. Derecha: mismo conjunto de puntos visto desde el ángulo que deja ver de canto los hiperplanos sobre los que caen todas las ternas.

Puede demostrarse que toda terna consecutiva de RANDU satisface exactamente  $x_{n+2} = 6x_{n+1} - 9x_n$  (mód  $2^{31}$ ), lo cual implica que los puntos  $(u_n, u_{n+1}, u_{n+2})$  caen sobre la familia de planos paralelos  $9u_n - 6u_{n+1} + u_{n+2} = k$  para  $k$  entero. Dentro del cubo  $[0, 1]^3$  esa expresión solo puede tomar unos 15 valores enteros distintos, así que las ternas de RANDU quedan confinadas a apenas 15 hiperplanos, en vez de llenar el cubo. Desde un ángulo de vista genérico esto no salta a la vista, pero basta rotar la cámara para verlo con total claridad. Esta es la falla que Marsaglia documentó en *Random Numbers Fall Mainly in the Planes*.

## 1.6. Comparación de velocidad, período, seguridad y casos de uso

| Algoritmo        | Velocidad (núm/s)        | Período                             | Seguro             | Uso típico                           |
|------------------|--------------------------|-------------------------------------|--------------------|--------------------------------------|
| LCG              | $\sim 1,05 \times 10^7$  | $2^{32}$ (buenos parámetros)        | No                 | Simulaciones, videojuegos            |
| Cuadrados medios | $\sim 4,4 \times 10^6$   | Corto e impredecible                | No                 | Interés histórico/didáctico          |
| Mersenne Twister | $\sim 1,9 \times 10^6$   | $2^{19937} - 1$                     | No                 | Simulación científica, <b>random</b> |
| Blum Blum Shub   | $\sim 1,0 \times 10^7$ * | Grande si $p, q$ son primos seguros | Sí (criptográfico) | Claves, cifrado de flujo             |
| RANDU            | $\sim 1,2 \times 10^7$   | $2^{29}$ (defectuoso)               | No                 | Ninguno (obsoleto)                   |

Cuadro 1: Comparación de los cinco algoritmos. Las velocidades se midieron generando 100,000 números en la misma máquina. (\*) La velocidad de Blum Blum Shub depende fuertemente del tamaño de los primos: aquí se usaron primos pequeños por rapidez, no primos criptográficamente seguros.

Mersenne Twister resulta más lento que LCG y RANDU porque cada salida involucra varias operaciones de temple además de la recurrencia de estado, mientras que LCG y RANDU son una sola multiplicación y suma módulo  $m$ . Blum Blum Shub, pese a ser el único diseñado con garantías criptográficas, aquí resulta rápido solo porque se usaron primos de unas pocas decenas de bits; con primos de tamaño criptográfico real (2048 bits o más) sería órdenes de magnitud más lento que los demás, ya que ese costo computacional es precisamente lo que sostiene su seguridad.

## 1.7. Comparación con las librerías estándar de Python

Se comparó la implementación propia de Mersenne Twister contra `random.Random` (que también usa MT19937 internamente, implementado en C) y contra `secrets.SystemRandom` (que obtiene aleatoriedad del sistema operativo, pensado para uso criptográfico). Se generaron 300,000 números con cada uno:

| Generador                     | Tiempo (s) | $\chi^2$ | $p$ -valor |
|-------------------------------|------------|----------|------------|
| Mersenne Twister propio       | 0.157      | 21.93    | 0.288      |
| <code>random</code> estándar  | 0.010      | 26.14    | 0.126      |
| <code>secrets</code> (CSPRNG) | 0.193      | 33.59    | 0.021      |

Cuadro 2: Comparación de velocidad y bondad de ajuste,  $N = 300,000$ .

La versión de `random` de la librería estándar es unas 15 veces más rápida que la implementación propia porque su ciclo interno está escrito en C, mientras que la nuestra corre en Python puro instrucción por instrucción. `secrets` resulta el más lento de los tres, un costo esperado: para ser criptográficamente seguro no puede depender de un estado interno predecible como el de un Mersenne Twister, así que consulta al sistema operativo en cada llamada. El  $p$ -valor bajo de `secrets` en esta corrida particular no indica ningún defecto real, es simplemente la variabilidad normal de una prueba de hipótesis: repitiendo el experimento con otra semilla del sistema el resultado cambia.

## 2. Fundamento de Monte Carlo: el estimador y su error

### 2.1. Insesgadez del estimador

The image shows two pages of handwritten notes on graph paper. The top page contains the following derivations:

$$I = \int_a^b f(x) dx$$
$$x = a + (b-a)u$$
$$dx = (b-a)du$$
$$I = \int_0^1 f(a + (b-a)u) (b-a) du$$
$$I = (b-a) \int_0^1 f(a + (b-a)u) du$$
$$\int_0^1 g(u) du = E[g(U)]$$
$$U \sim \text{Unif}(a, b)$$
$$I = (b-a) E[f(U)]$$
$$\hat{I}_N = \frac{b-a}{N} \sum_{i=1}^N f(U_i)$$
$$U_i \sim \text{Unif}(a, b)$$
$$E[\hat{I}_N] = E\left[\frac{b-a}{N} \sum_{i=1}^N f(U_i)\right]$$

The bottom page continues the derivation:

$$E[\hat{I}_N] = \frac{b-a}{N} \cdot N \cdot E[f(U)]$$
$$E[\hat{I}_N] = (b-a) E[f(U)]$$
$$I = (b-a) E[f(U)]$$
$$\text{---}$$
$$E[\hat{I}_N] = I$$

$\hat{I}_N$  es un estimador insesgado de  $I$ .

Figura 8: Derivación a mano de  $I = (b-a)E[f(U)]$  y de  $E[\hat{I}_N] = I$ .

## 2.2. Varianza del estimador y tasa de convergencia

Handwritten derivation on a grid background:

$$\hat{I}_N = \frac{b-a}{N} \sum_{i=1}^N f(U_i)$$

$$\text{Var}(\hat{I}_N) = \text{Var}\left(\frac{b-a}{N} \sum_{i=1}^N f(U_i)\right)$$

$$\text{Var}(\hat{I}_N) = \frac{(b-a)^2}{N^2} \text{Var}\left(\sum_{i=1}^N f(U_i)\right)$$

$$\text{Var}(\hat{I}_N) = \frac{(b-a)^2}{N^2} \sum_{i=1}^N \text{Var}(f(U_i))$$

$$\sigma^2 = \text{Var}(f(U))$$

$$\text{Var}(\hat{I}_N) = \frac{(b-a)^2}{N^2} \cdot N \sigma^2$$

$$\text{Var}(\hat{I}_N) = \frac{(b-a)^2}{N} \sigma^2$$

$$\text{ec}(\hat{I}_N) = \sqrt{\text{Var}(\hat{I}_N)} \rightarrow \text{ec}(\hat{I}_N) = \sqrt{\frac{(b-a)^2 \sigma^2}{N}}$$

$$\text{ec}(\hat{I}_N) = \frac{(b-a) \sigma}{\sqrt{N}} \rightarrow \text{ec}(\hat{I}_N) = O(N^{-1/2})$$

Figura 9: Derivación a mano de  $\text{Var}(\hat{I}_N)$  y de por qué el error estándar decae como  $O(N^{-1/2})$ .

Esta tasa no depende de la dimensión del problema porque la Ley de los Grandes Números y el Teorema del Límite Central se aplican sobre el promedio de variables aleatorias escalares  $f(U_i)$ , sin importar de qué espacio provienen los  $U_i$ : lo único que entra en la varianza del estimador es  $\text{Var}(f(U))$ , un solo número real, nunca la dimensión de  $U$ . Un integrando en  $d = 20$  dimensiones

tiene exactamente la misma tasa  $O(N^{-1/2})$  que uno en  $d = 1$ ; lo que cambia con  $d$  es qué tan grande es esa varianza, no la forma en que decae con  $N$ . Esta observación es la que hace útil a Monte Carlo en la Sección 4: los métodos de cuadratura determinística sí empeoran con la dimensión, Monte Carlo no.

### 2.3. Intervalo de confianza del 95 %

$$\frac{\hat{I}_N - I}{\text{ee}(\hat{I}_N)} \xrightarrow{d} \mathcal{N}(0, 1)$$

$$P\left(-1.96 \leq \frac{\hat{I}_N - I}{\text{ee}(\hat{I}_N)} \leq 1.96\right) \approx 0.95$$

$$P(\hat{I}_N - 1.96 \text{ee}(\hat{I}_N) \leq I \leq \hat{I}_N + 1.96 \text{ee}(\hat{I}_N)) \approx 0.95$$

$$\hat{I}_N \pm 1.96 \text{ee}(\hat{I}_N)$$

$$\text{ee}(\hat{I}_N) = \frac{(b-a)\hat{\sigma}}{\sqrt{N}}$$

$\hat{\sigma}^2$  es la varianza muestral de  $f(u_1), f(u_2), \dots, f(u_N)$

$$\hat{I}_N \pm 1.96 \text{ee}(\hat{I}_N)$$

Figura 10: Construcción del intervalo de confianza del 95 % para  $\hat{I}_N$  usando el TLC.

De aquí en adelante todo estimador Monte Carlo reportado en este documento viene acompañado de su intervalo de confianza del 95 %, calculado como  $\hat{I}_N \pm 1,96 \cdot \widehat{ee}(\hat{I}_N)$ , con  $\widehat{ee}(\hat{I}_N)$  el error estándar estimado a partir de la muestra.

### 3. Calentamiento: integrales en una dimensión

Se implementó el estimador de la sección anterior en `src/montecarlo/estimator.py` y se aplicó a las dos integrales del enunciado, muestreando  $U \sim \text{Unif}(a, b)$  con la semilla 20250922.

#### 3.1. Integral (a): $\int_0^1 \sin(\pi x) dx$

El valor real es  $2/\pi \approx 0,636620$ . Con  $N = 1,000,000$  se obtuvo  $\hat{I}_N = 0,636421$ , con intervalo de confianza del 95 % (0,635817, 0,637024), que en efecto contiene el valor real. El error absoluto fue de apenas 0,000199.

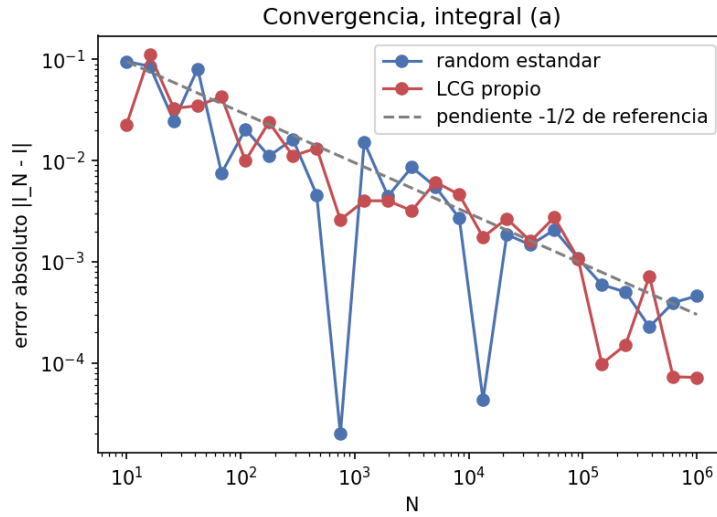


Figura 11: Error absoluto  $|\hat{I}_N - I|$  contra  $N$  en escala log-log, integral (a).

El ajuste de una recta a  $\log_{10} |\hat{I}_N - I|$  contra  $\log_{10} N$  dio una pendiente de  $-0,454$ , cercana a la tasa teórica  $-1/2$  derivada en la Sección anterior. La diferencia con  $-0,5$  exacto es simplemente ruido de una sola trayectoria: el error absoluto de una corrida particular no decae monótonamente, oscila alrededor de la tendencia  $O(N^{-1/2})$ , y una pendiente ajustada sobre una sola realización rara vez da exactamente  $-0,5$ .

#### 3.2. Integral (b): $\int_0^2 \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx$

El valor real es  $\Phi(2) - \Phi(0) \approx 0,477250$ . Con  $N = 1,000,000$  se obtuvo  $\hat{I}_N = 0,477237$ , IC 95 % (0,476786, 0,477689), error absoluto 0,000013.

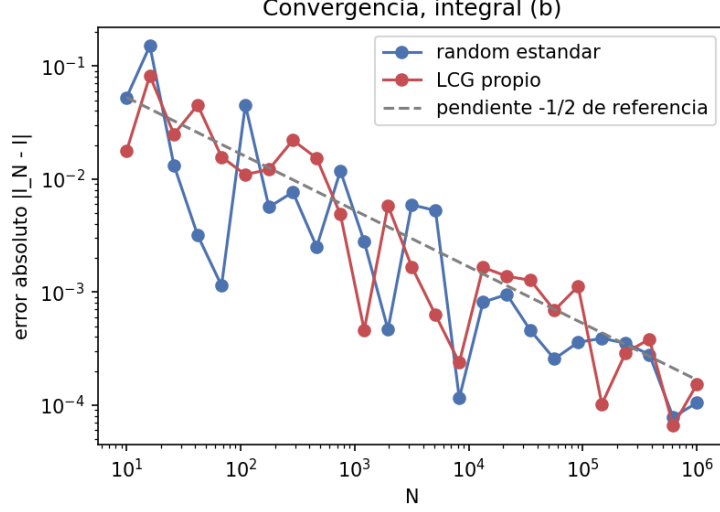


Figura 12: Error absoluto  $|\hat{I}_N - I|$  contra  $N$  en escala log-log, integral (b).

Aquí la pendiente ajustada fue  $-0,493$ , todavía más cercana a  $-1/2$ . Ambos casos confirman empíricamente la tasa de convergencia derivada analíticamente.

### 3.3. Comparación entre generadores

Las dos figuras anteriores incluyen, además del generador `random` estándar de Python, la misma integral estimada con el LCG propio de la Parte 1 (Sección 1.1), usando en cada punto  $N$  un bloque nuevo de valores de esa misma secuencia. Las dos curvas de error son estadísticamente indistinguibles entre sí y ambas siguen la tendencia  $O(N^{-1/2})$ : para una integral de buen comportamiento en una dimensión, un LCG con parámetros razonables rinde tan bien como el generador de la librería estándar. La diferencia entre generadores solo se vuelve relevante cuando el generador tiene defectos estructurales serios, como se ve con RANDU en la Sección ??.

## 4. Alta dimensión y la maldición de la dimensionalidad

Se implementó un estimador Monte Carlo para el volumen de la bola unitaria  $V_d$  en  $d$  dimensiones, muestreando puntos uniformes en el cubo  $[-1, 1]^d$  con el LCG propio y contando la fracción que cae dentro de la bola. Como referencia se usó el valor cerrado  $V_d = \pi^{d/2}/\Gamma(d/2 + 1)$ , y como contraparte determinística se implementó una cuadratura por rejilla, evaluando el indicador en el centro de cada celda de una malla de  $m$  subdivisiones por eje, con  $m = \lfloor N^{1/d} \rfloor$  para igualar aproximadamente el presupuesto de puntos  $N = 1,000,000$  entre ambos métodos.

| $d$ | $V_d$ real | Estimado MC (IC 95 %)      | Error absoluto MC |
|-----|------------|----------------------------|-------------------|
| 2   | 3.14159    | 3.14244 (3,13922, 3,14566) | 0.000847          |
| 5   | 5.26379    | 5.26288 (5,23963, 5,28613) | 0.000909          |
| 10  | 2.55016    | 2.46374 (2,36542, 2,56207) | 0.086420          |
| 20  | 0.025807   | 0.000000 (0, 3,14573)      | 0.025807          |

Cuadro 3: Estimación Monte Carlo de  $V_d$  con  $N = 1,000,000$  puntos, semilla base 20250923.

En  $d = 20$  no se registró ni un solo acierto: de los  $10^6$  puntos muestreados, se esperaban en promedio  $N \cdot V_{20}/2^{20} \approx 1,000,000 \times 2,46 \times 10^{-8} \approx 0,025$  aciertos, así que obtener cero es exactamente lo esperado, no una falla del código. Con cero aciertos el intervalo de confianza normal de la Sección 2.3 degenera (varianza estimada cero), así que se reporta en su lugar una cota superior aproximada con la regla de tres,  $3/N$ , escalada por  $2^d$ .

#### 4.1. Comparación contra cuadratura determinística

| $d$ | $m$ (rejilla) | Puntos rejilla | Estimado rejilla | Error absoluto rejilla |
|-----|---------------|----------------|------------------|------------------------|
| 2   | 1000          | 1,000,000      | 3.14182          | 0.000231               |
| 5   | 16            | 1,048,576      | 5.31152          | 0.047734               |
| 10  | 4             | 1,048,576      | 1.00000          | 1.550160               |
| 20  | 2             | 1,048,576      | 0.00000          | 0.025807               |

Cuadro 4: Cuadratura por rejilla con presupuesto de puntos comparable al de Monte Carlo.

Con un presupuesto de puntos similar, la rejilla le gana a Monte Carlo en  $d = 2$  (su error es casi 4 veces menor), pero para  $d = 5$  ya es Monte Carlo el que gana claramente, y para  $d = 10$  la rejilla es catastrófica: con solo  $m = 4$  subdivisiones por eje, la malla es tan gruesa que ni siquiera logra resolver la curvatura de la esfera y termina estimando el volumen de un solo cubito central, muy lejos del valor real. En  $d = 20$  ambos métodos fallan por completo con este presupuesto, pero por razones distintas: la rejilla porque  $m = 2$  es una resolución absurda para captar una esfera, y Monte Carlo porque la probabilidad de acierto es demasiado baja para el número de muestras disponible.

| $d$ | Puntos necesarios en rejilla ( $m = 10$ ) |
|-----|-------------------------------------------|
| 2   | 100                                       |
| 5   | 100,000                                   |
| 10  | 10,000,000,000                            |
| 20  | 100,000,000,000,000,000,000               |

Cuadro 5: Número de puntos  $m^d$  que necesitaría la rejilla para mantener 10 subdivisiones por eje.

El número de puntos de la rejilla crece como  $m^d$ : exponencial en la dimensión. Mantener una resolución fija de 10 subdivisiones por eje es factible en  $d = 2$  (cien puntos) pero ya es completamente inalcanzable en  $d = 20$  ( $10^{20}$  puntos, más que cualquier cómputo razonable podrá evaluar jamás). Esta es la esencia de la maldición de la dimensionalidad: los métodos que discretizan cada eje por separado sufren un costo que se multiplica, no se suma, con cada dimensión nueva. Monte Carlo, en cambio, mantiene su tasa de error  $O(N^{-1/2})$  sin importar  $d$  (Sección 2.2), lo que explica por qué termina siendo la única opción viable en dimensiones altas, aun cuando su desempeño relativo en  $d = 2$  sea mediocre frente a una rejilla fina.

## 4.2. Por qué el muestreo uniforme se vuelve ineficiente

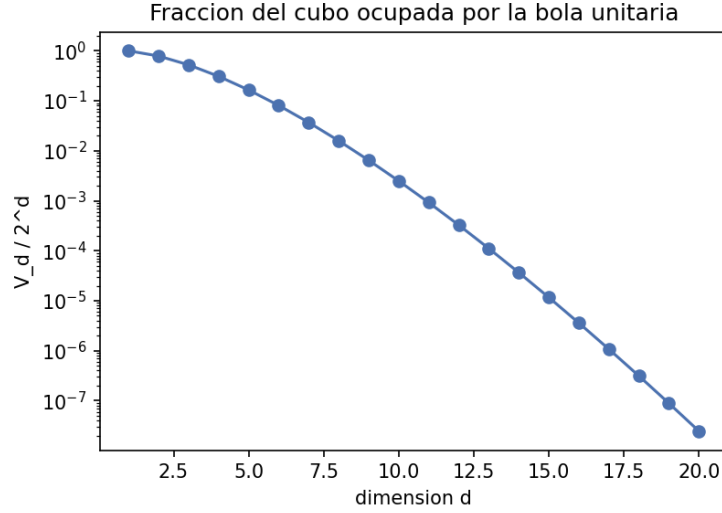


Figura 13: Fracción del volumen del cubo  $[-1, 1]^d$  ocupada por la bola unitaria,  $V_d/2^d$ , en escala logarítmica.

La fracción  $V_d/2^d$  decae exponencialmente con  $d$ : en  $d = 2$  la bola todavía ocupa un 79% del cuadrado, pero en  $d = 20$  ocupa apenas  $2,46 \times 10^{-8}$  del cubo. Esto revela que, en alta dimensionalidad, casi todo el volumen de un hipercubo vive cerca de sus esquinas y muy lejos del centro: la masa se concentra en las "puntas" del cubo, una manifestación geométrica directa de la maldición de la dimensionalidad. Para un estimador Monte Carlo esto es letal si se muestrea ingenuamente en el cubo entero, porque la enorme mayoría de los puntos generados caen en la región irrelevante (fuera de la bola) y aportan cero información sobre la cantidad que se quiere medir; se necesitan cantidades astronómicas de muestras para obtener siquiera unos pocos aciertos útiles. Esta es precisamente la motivación detrás de las técnicas de reducción de varianza y de muestreo por importancia: en vez de muestrear a ciegas donde casi nunca hay señal, conviene concentrar el muestreo donde el integrando realmente aporta.

## 5. Reducción de varianza

El error de Monte Carlo decae como  $\sigma/\sqrt{N}$ , así que además de aumentar  $N$  la única otra vía es reducir  $\sigma$ . Se implementaron y compararon dos técnicas, variables antitéticas y variables de control, aplicadas a las integrales de la Sección 3, usando en todos los casos  $N = 2,000,000$  muestras uniformes con semilla 20250924.

### 5.1. Variables antitéticas

La idea es reemplazar cada muestra  $U_i$  por el promedio  $Y_i = \frac{1}{2}(f(U_i) + f(a + b - U_i))$ : si  $f(U)$  y  $f(a + b - U)$  están negativamente correlacionadas, promediarlas reduce la varianza. Comparando a igualdad de número total de evaluaciones de  $f$  ( $N$  evaluaciones simples contra  $N/2$  pares), el factor de reducción de varianza es  $\text{Var}(f(U))/(2 \text{Var}(Y))$ .

Para la integral (a),  $f(x) = \sin(\pi x)$  en  $[0, 1]$ , el par antitético es  $x$  y  $1 - x$ , y como  $\sin(\pi(1 - x)) = \sin(\pi x)$  exactamente, ambos miembros del par valen siempre lo mismo:  $Y_i = f(U_i)$  sin ninguna

reducción de varianza, solo el doble de costo. El experimento lo confirma:  $\text{Var}(f(U)) = 0,094657$  y  $\text{Var}(Y) = 0,094657$  (idénticas), dando un factor de reducción de 0,5, es decir, antitéticas resulta *el doble de caro* que Monte Carlo simple para la misma precisión en este caso.

Para la integral (b),  $f$  es la densidad normal estándar en  $[0, 2]$ , monótona decreciente en ese intervalo. Aquí el par antitético  $x, 2 - x$  sí está negativamente correlacionado:  $\text{Var}(f(U)) = 0,013253$  contra  $\text{Var}(Y) = 0,0000182$ , dando un factor de reducción de 363, equivalente a necesitar 363 veces menos muestras para el mismo error con Monte Carlo simple.

## 5.2. Variables de control

La idea es restar de  $f(U)$  un múltiplo de una variable  $g(U)$  de media conocida  $\mu_g$ : el estimador controlado  $H = f(U) - c(g(U) - \mu_g)$  sigue siendo insesgado para cualquier  $c$ , y con  $c^* = \text{Cov}(f, g)/\text{Var}(g)$  se minimiza  $\text{Var}(H) = \text{Var}(f) - \text{Cov}(f, g)^2/\text{Var}(g)$ . Se probaron dos controles para la integral (a):

- $g(x) = x$ , con  $\mu_g = 0,5$ : la correlación muestral con  $\sin(\pi x)$  resultó  $-0,0002$ , esencialmente cero (tiene sentido:  $\sin(\pi x)$  es simétrico alrededor de  $x = 0,5$  mientras que  $x$  es una función lineal creciente, así que no hay relación lineal entre ambas). El factor de reducción fue 1,0000: este control no ayuda en absoluto.
- $g(x) = x(1 - x)$ , con  $\mu_g = 1/6$ : esta función también es una joroba simétrica que se anula en los extremos, con una forma mucho más parecida a  $\sin(\pi x)$ . La correlación muestral resultó 0,9984, casi perfecta, y el factor de reducción fue 318.

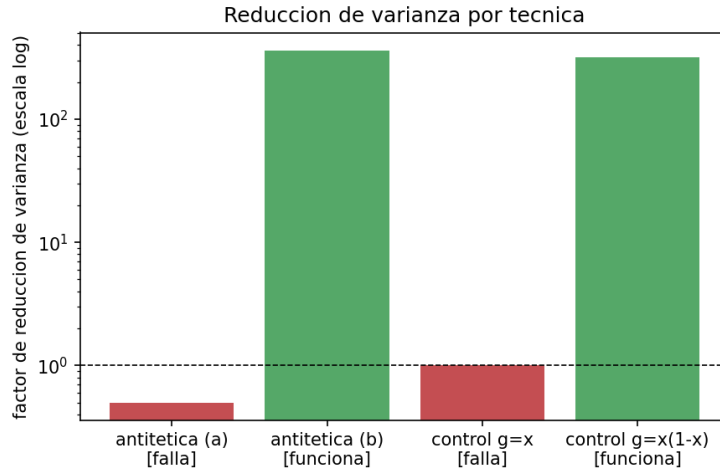


Figura 14: Factor de reducción de varianza logrado por cada técnica (escala logarítmica; la línea punteada marca el punto de equilibrio con Monte Carlo simple).

## 5.3. ¿Siempre ayuda, o depende del integrando?

Los cuatro experimentos responden la pregunta de forma inequívoca: depende por completo del integrando y de qué tan bien elegida esté la técnica para su forma particular. Variables antitéticas solo funciona cuando  $f$  tiene la simetría correcta (monotonía en el intervalo, como en (b)); aplicada a una función simétrica alrededor del punto medio del intervalo, como (a), el par antitético es

redundante y el método empeora el desempeño respecto a Monte Carlo simple. Variables de control solo ayuda si se elige un control realmente correlacionado con el integrando: un control lineal simple no sirvió de nada para (a), mientras que un control con la forma correcta (una joroba simétrica) redujo la varianza en más de dos órdenes de magnitud. La lección práctica es que ninguna de estas técnicas es una mejora automática: hay que entender la forma del integrando antes de elegir cuál aplicar, y una elección descuidada puede incluso ser contraproducente.